

CODE ARCHAEOLOGIST

Full Product & Engineering Scope

An open-source AI system that reconstructs not only what a codebase does, but why it evolved into its current architecture.

Backend	Database	Frontend	AI
NestJS + TypeScript	PostgreSQL	React + TypeScript	Provider abstraction Ollama-first

Build target: a production-quality, self-hostable repository intelligence platform with Git history, AST analysis, dependency graph, Code DNA, impact analysis, AI investigation, CLI/SDK, integrations, security, testing and operations.

Version 1.0 • September 2026

1. Product Definition

Code Archaeologist turns a Git repository into an evidence-backed historical knowledge graph. It answers not only “what does this code do?” but “why does it exist, who changed it, what changed around it, what depends on it, and what is the likely impact of changing it?”

Core flow

Git repository → ingestion → Git history + AST → normalized symbols → dependency graph → metrics/risk → evidence index → AI investigation → cited explanation.

Primary principles

- Evidence before AI: deterministic facts are retrieved before language-model synthesis.
- AI never silently changes repositories in v1.
- Unsupported historical claims must be labeled as unknown/uncertain.
- Local/self-hosted operation is a first-class requirement.
- Modular monolith + workers first; extract services only when scale requires it.

2. Users, Use Cases & Scope

User	Need	Features
Senior developer	Understand unfamiliar code	Explorer, Code DNA, history, Q&A;, impact
Tech lead	Detect architecture drift	Hotspots, coupling, evolution, risk
New engineer	Learn system history	Historical Q&A;, timelines, evidence
Maintainer	Assess risky changes	Blast radius, churn, ownership, tests

In scope for v1

- TypeScript/JavaScript support first.
- Git commit/file/branch history.
- AST symbols and relationships.
- Dependency and knowledge graph.
- Code DNA profiles.
- Complexity/churn/coupling metrics.
- Risk and impact analysis.
- Evidence-grounded AI investigation.
- React dashboard and graph.
- GitHub enrichment, CLI, SDK and Docker deployment.

Explicitly deferred

- Autonomous code modification.
- Every programming language.
- Full APM/incident-management replacement.
- Microservice architecture.

- Marketplace before the plugin contract is stable.

3. Technology Stack

Layer	Technology	Purpose
API	NestJS + TypeScript	REST API, auth, orchestration, policies
DB	PostgreSQL	Core data, graph edges, search, analytics
Queue	Redis + BullMQ	Long-running analysis and retryable jobs
Parser	Tree-sitter + language adapters	AST and structural facts
Git	Git CLI / compatible library	History, diffs, branches, blame
AI	Ollama-first provider abstraction	Local inference and optional hosted providers
Web	React + TypeScript + Tailwind	Dashboard, explorer, graph, investigation
Editor	Monaco	Source and evidence preview
Graph UI	React Flow or equivalent	Interactive dependency architecture
Deployment	Docker Compose; Kubernetes later	Self-hosting

4. Repository Architecture

Recommended monorepo

apps/ api/ worker/ web/ packages/ core/ parser/ git/ ai/ shared/ sdk/ cli/ infra/ docker/ docs/ examples/

NestJS modules

auth • users • workspaces • repositories • git • analysis • parsers files • symbols • graph • metrics • risks • insights • investigations ai • embeddings • integrations • webhooks • reports • notifications • audit

Keep controllers thin. Business rules live in services/domain components. Git, parser and AI providers must be behind interfaces so implementations can change independently.

5. PostgreSQL Data Model

Core tables

Table	Important fields / purpose
users	id, email, password_hash, name, status, timestamps
workspaces	id, name, slug, owner_id, settings
workspace_members	workspace_id, user_id, role, status
repositories	id, workspace_id, url, provider, default_branch, settings, status
repository_credentials	id, repository_id, encrypted_secret_ref, type, rotation data
branches	repository_id, name, head_commit_id, is_default
commits	repository_id, sha UNIQUE, message, author, dates, parents
commit_files	commit_id, file_id, change_type, old/new path, additions/deletions

Table	Important fields / purpose
files	repository_id, path UNIQUE, language, hash, size, first/last revision
symbols	file_id, parent_symbol_id, kind, name, qualified_name, lines, ast_hash
symbol_versions	symbol_id, commit_id, content_hash, lines, change_type, metrics
graph_edges	source/target type+id, edge_type, confidence, first/last revision
developers	repository_id, canonical_name, canonical_email
issues	external_id, title, body, status, author, dates
pull_requests	external_id, title, body, state, author, merged_at
reviews	pull_request_id, reviewer, state, body, submitted_at
analysis_runs	repository_id, revision, type, status, progress, times, error
analysis_tasks	analysis_run_id, task_type, status, attempts, error
metrics_snapshots	repository_id, revision, scope, metrics_json
risk_scores	subject_type/id, score, level, factors_json, calculated_at
documents	repository_id, source_type/id, title, content, metadata
document_chunks	document_id, chunk_index, content, token_count, embedding
investigations	repository_id, user_id, question, status, model
investigation_evidence	investigation_id, source_type/id, excerpt, relevance, citation
investigation_messages	investigation_id, role, content, token usage
integrations	workspace_id, provider, encrypted credential reference, settings
webhook_events	integration_id, provider_event_id, type, hash, status
audit_logs	workspace_id, user_id, action, resource, metadata, timestamp

Indexing rules

- Unique commit SHA per repository.
- Unique file path per repository.
- Indexes on repository/time, symbol qualified_name, graph source/target, analysis status, investigation time.
- Use JSONB for variable metadata, not as a replacement for queryable columns.
- Use pgvector only when semantic embeddings are enabled.

6. Authentication, Authorization & Workspace

- Email/password authentication with secure password hashing.
- Access/refresh session strategy with revocation.
- Workspace roles: Owner, Admin, Analyst, Viewer.
- Object-level authorization for every repository/resource.
- Member invitation and removal.
- Workspace archive/delete workflow.

- Audit security-sensitive operations.
- Rate-limit login and sensitive endpoints.
- Optional GitHub OAuth later.

7. Repository Onboarding

- 1 Create repository record.
- 2 Validate URL/provider.
- 3 Store credentials only through encrypted secret abstraction.
- 4 Clone/fetch in isolated worker directory.
- 5 Detect default branch and current revision.
- 6 Create analysis run.
- 7 Show live progress.
- 8 Clean temporary working directory after processing.
- 9 Allow manual re-sync and branch/revision selection.

Supported inputs

- Public HTTPS Git repositories.
- Private repositories through provider credentials.
- Local repository path through CLI.
- Ignore .gitignore plus product ignore rules.
- Detect generated/vendor/binary files and skip them by default.

8. Git History Engine

- Commit metadata and parent relationships.
- Added/modified/deleted/renamed files.
- Diff statistics and patch references.
- Branch and tag discovery.
- File history.
- Blame evidence for selected lines.
- Author normalization.
- Rename/copy detection with confidence.
- Merge commit handling.
- Incremental sync from the last indexed revision.

Commit-to-symbol linking

Use changed file + changed line ranges + symbol boundaries to associate commits with symbols. Store mapping confidence. Heuristic mappings must never be represented as absolute certainty.

9. AST & Source Analysis

Phase 1 language: TypeScript/JavaScript. Build a parser adapter interface so additional languages can be added without changing graph/AI layers.

- File and language detection.
- Classes, interfaces, enums, functions, methods, variables/constants where useful.
- Imports/exports.
- Calls/references where resolvable.
- Inheritance/implementation.
- Decorators/annotations.
- Framework-aware route/controller/provider extraction as optional adapters.
- LOC, nesting, complexity and other deterministic metrics.
- Parser error collection without aborting the complete repository.

Parser contract

```
parse(file) → symbols + relations + diagnostics  
normalize(ast facts) → stable symbol records  
compare(previous,current) → structural changes
```

10. Knowledge Graph

Nodes

- Repository, branch, commit, file, symbol, module, package, developer, pull request, issue, review, test and endpoint.

Edges

- contains, imports, exports, calls, references, extends, implements, modifies, authored, reviewed, depends_on, tests, related_to.

Every edge should store type, source/target, confidence, source of evidence and first/last observed revision. This makes the graph historical rather than merely a snapshot.

11. Code DNA

For every important file/symbol/module, expose

- First seen / created revision.
- Last changed revision.
- Contributors.
- Change frequency/churn.
- Fan-in/fan-out.
- Complexity.
- Dependency count.
- Related commits/PRs/issues.
- Risk score and factor breakdown.
- Evidence confidence.
- Historical timeline.

Example profile

12. Architecture Intelligence

- Module dependency map.
- Circular dependency detection.
- High fan-in/fan-out modules.
- High churn + high complexity hotspots.
- Knowledge silos/author concentration.
- Architecture rule engine.
- Historical architecture snapshots.
- Drift detection between revisions.
- Trend charts.

Example rules

- Controller must not directly access database adapter.
- Payments module must not import internal authentication implementation.
- Public module cannot depend on a configured private module.
- No circular dependency between bounded domains.

13. Risk Scoring & Impact Analysis

Use an explainable weighted score first. Store each factor so users can understand why a subject is risky.

Factor	Signal
Fan-in	Number of consumers
Fan-out	Number of dependencies
Churn	Historical change frequency
Complexity	Structural complexity
Ownership	Contributor concentration
Defect evidence	Related issue/PR evidence when available
Architecture violation	Configured rule failures
Recency	Recent change signal

Impact algorithm

- 1 Resolve selected file/symbol.
- 2 Traverse direct consumers and dependencies.
- 3 Traverse configurable depth.
- 4 Overlay tests/endpoints/modules where available.
- 5 Overlay churn/risk metrics.

- 6 Return deterministic blast radius.
- 7 Let AI explain the result; do not let AI invent the dependency set.

14. AI Investigation & RAG

Provider interface

```
interface LlmProvider { chat(request): Promise; embed?(request): Promise; health(): Promise; }
```

Retrieval layers

- SQL/entity retrieval for exact symbols/files/commits.
- Graph traversal for dependencies and impact.
- PostgreSQL full-text retrieval for commit/issue/document text.
- Temporal filters for historical questions.
- pgvector semantic retrieval for documents/chunks when enabled.

Investigation pipeline

Question → intent/entity resolution → graph facts → Git evidence → integration evidence → ranking → context construction → LLM synthesis → citation validation → answer + confidence.

Supported question families

- Why does this function/file exist?
- Who introduced this behavior?
- When did architecture change?
- What changed around this symbol?
- What depends on this?
- What happens if I remove/change this?
- Which modules are risky?
- How has payments/auth/etc. evolved?

Trust requirements

Repository text, issue text and commit messages are untrusted data. Treat them as data, not instructions. AI answers must distinguish direct facts from inferred explanations and explicitly state when evidence is insufficient.

15. Frontend Scope

Screen	Requirements
Auth	Login, registration, session/errors
Workspace	Members, repositories, recent activity
Repository Overview	Health, languages, revision, analysis status
Explorer	Tree, search, source, symbols, history
Architecture	Interactive graph, filters, grouping, depth
Code DNA	Metrics, timeline, authors, dependencies, risk, evidence

Screen	Requirements
Commit Explorer	Commit metadata, changed files, diff, linked symbols
Evolution	Historical snapshots and trends
Impact	Blast-radius graph and deterministic explanation
Investigation	Chat, streaming answer, evidence/citations, source preview
Hotspots	Risk/churn/complexity/coupling filters
Settings	AI, analysis, integrations, retention, members
Operations	Queue/run failures and system health for self-hosting

UX rules

- Always show analysis stage/progress.
- Never hide citations behind a generic AI response.
- Graph filtering/aggregation is mandatory for large repositories.
- Every screen has loading/empty/error states.
- Source evidence links to file and line when available.
- Show confidence and limitations.

16. REST API

Use `/api/v1`. Long operations return run IDs and are processed by workers.

Area	Endpoints
Auth	POST <code>/auth/register</code> , <code>/login</code> , <code>/refresh</code> , <code>/logout</code> ; GET <code>/me</code>
Workspaces	CRUD <code>/workspaces</code> ; members endpoints
Repositories	CRUD <code>/repositories</code> ; POST <code>/:id/sync</code> ; status
History	branches, commits, commit details, file history
Source	files, symbols, symbol history, source previews
Graph	graph, dependencies, dependents, cycles
Analysis	start, status, tasks, cancel
Insights	health, hotspots, risks, evolution
Investigation	create investigation, messages, evidence
Integrations	CRUD integrations; provider webhooks
Reports	create/get report
Audit	workspace audit log

API standards

- Global DTO validation.
- Consistent error code/message/request ID.

- Pagination for large collections.
- Object authorization on every resource.
- Idempotent webhook processing.
- OpenAPI documentation.
- Rate limits on authentication/investigation/webhooks.

17. Queue & Worker Design

BullMQ queues

- repository-sync
- analysis-run
- git-history
- ast-parse
- graph-build
- metrics
- risk
- embedding
- integration-sync
- report-generation
- cleanup

Worker requirements

- Retry with backoff.
- Dead-letter handling.
- Idempotent jobs.
- Progress events.
- Cancellation checks.
- Bounded concurrency.
- CPU/RAM/disk/time limits.
- Persistent task status in PostgreSQL.

Never execute repository code during normal analysis. Parsing and static inspection must be the default.

18. GitHub Integration

- Connect GitHub account/app.
- Repository metadata.
- Pull requests.
- Issues.
- Reviews/comments where permitted.
- Link PRs to commits.

- Link issue/PR text to changed files/symbols with confidence.
- Webhook verification.
- Replay protection/idempotent event processing.
- Incremental synchronization.

Later: GitLab and Bitbucket adapters using the same integration contract.

19. CLI & SDK

CLI commands

```
code-archaeologist init code-archaeologist analyze code-archaeologist status code-archaeologist ask "Why does this module exist?" code-archaeologist impact path/to/file.ts code-archaeologist hotspots code-archaeologist report --format json code-archaeologist doctor
```

CLI must support local repositories, self-hosted API mode, JSON output, useful exit codes, progress output and zero secret leakage.

SDK

- Typed repository/analysis/investigation clients.
- Authentication helpers.
- Pagination.
- Webhook signature verification.
- Versioning and compatibility policy.

20. Security & Privacy

- Argon2id or equivalent secure password hashing.
- Encrypted repository credentials; encryption keys outside DB.
- Never log tokens/passwords/secrets.
- Isolated repository working directories.
- No arbitrary shell execution.
- Resource limits for untrusted repositories.
- Prompt-injection defenses.
- Provider disclosure when source is sent to hosted AI.
- Workspace isolation/RBAC.
- Webhook signature validation.
- Audit logs.
- Retention and deletion controls.
- Dependency/container vulnerability scanning.

Threat model

- Malicious repository prompt injection.
- Malicious parser input.
- Compromised Git token.

- Cross-workspace leakage.
- Huge graph traversal denial of service.
- AI provider data leakage.
- Webhook forgery/replay.
- Sensitive source in logs/reports.

21. Testing Strategy

Test layer	What to test
Unit	Services, parsers, graph algorithms, risk calculations, DTOs
Integration	PostgreSQL repositories/services, Redis/BullMQ workflows
Parser fixtures	Real TS/JS syntax, malformed files, edge cases
E2E	Register → repo → analysis → explorer → investigation
Security	RBAC, object access, auth abuse, webhook verification
Performance	Large history, many files/symbols, graph traversal, concurrent jobs
AI evaluation	Golden questions, evidence accuracy, hallucination/unsupported-claim tests
Recovery	Worker crash, Redis/DB restart, partial analysis, provider timeout

AI benchmark

Maintain 50–200 benchmark questions against public repositories. Evaluate evidence precision, unsupported claim rate, completeness and latency. Keep expected evidence references rather than relying only on natural-language answer matching.

22. Observability & Operations

- Structured logs with request_id, workspace_id, repository_id and analysis_run_id.
- Metrics for queue depth, parse failures, analysis duration, throughput, DB latency, AI latency/token usage.
- Liveness/readiness health checks.
- Optional OpenTelemetry tracing.
- Dead-letter queue and retry dashboards.
- Operator diagnostics without exposing source/secrets.
- Backup/restore monitoring.

23. Local Development & Docker

Initial Docker Compose

```
services: postgres redis api worker web ollama # optional profile
```

- .env.example with no real credentials.
- Startup environment validation.
- Database migrations.
- Seed only safe demo data.

- Health checks.
- Document local CPU/RAM expectations for Ollama.
- Clean development setup from a fresh checkout.

24. Production Deployment

- HTTPS reverse proxy.
- Managed PostgreSQL recommended for hosted installs.
- Redis persistence/HA according to deployment size.
- Separate API and worker scaling.
- Persistent storage only for intentionally retained artifacts.
- Automated database backups.
- Tested restore procedure.
- Resource quotas.
- Monitoring and alerting.
- Secure secret injection.

Small self-hosted installation

One machine can run Docker Compose for small repositories. Larger installations should scale workers horizontally and use managed PostgreSQL/Redis where appropriate.

25. Open-Source Repository & Governance

Required files

```
README.md LICENSE CONTRIBUTING.md CODE_OF_CONDUCT.md SECURITY.md CHANGELOG.md .env.example docker-compose.yml
.github/workflows/ .github/ISSUE_TEMPLATE/ .github/PULL_REQUEST_TEMPLATE.md docs/ examples/
```

- Use MIT or Apache-2.0 after checking dependency/license compatibility.
- Publish architecture decision records.
- Public roadmap.
- Good-first-issue/help-wanted labels.
- Security vulnerability reporting process.
- Contribution and PR standards.
- Privacy/telemetry policy.
- Versioned releases.

26. Documentation Deliverables

- Quick start.
- Architecture.
- Installation/Docker.
- Configuration reference.
- CLI reference.

- REST/OpenAPI reference.
- Parser adapter guide.
- Git provider guide.
- AI provider guide.
- RAG/evidence model.
- Security model.
- Troubleshooting.
- Development setup.
- Testing/evaluation.
- Contribution guide.
- FAQ/limitations.

27. Step-by-Step Build Roadmap

Phase 0 — Foundation

- Create GitHub repo and license.
- Set monorepo structure.
- Configure TypeScript, lint, format, tests, CI.
- Create NestJS API/worker and React web apps.
- Docker Compose PostgreSQL/Redis.
- Environment validation.
- Migration framework.

Phase 1 — Identity

- Users, auth, sessions.
- Workspaces and memberships.
- RBAC.
- Audit events.
- Basic dashboard.

Phase 2 — Repository ingestion

- Repository CRUD.
- Credential abstraction.
- Clone/fetch worker.
- Analysis runs/tasks.
- Progress streaming.
- Repository status UI.

Phase 3 — Git engine

- Commits/branches/diffs.

- Changed files.
- Author normalization.
- File history.
- Incremental sync.
- Commit explorer.

Phase 4 — AST engine

- TS/JS parser adapter.
- File/language detection.
- Symbols.
- Imports/exports.
- Calls/references.
- Metrics.
- Fixtures/regressions.

Phase 5 — Graph

- Graph schema.
- Edge extraction.
- Dependency traversal.
- Cycles.
- Interactive architecture map.

Phase 6 — Code DNA

- Symbol versions.
- Churn/complexity/coupling.
- Risk score.
- Hotspots.
- Code DNA UI.

Phase 7 — Historical intelligence

- Evidence model.
- Commit/symbol linking.
- Historical resolver.
- Evolution timeline.
- Evidence API.

Phase 8 — AI investigation

- Provider interface.
- Ollama adapter.
- Structured/graph retrieval.
- Optional pgvector.

- Citations/confidence.
- AI benchmark.

Phase 9 — GitHub

- Connection.
- PR/issue/review ingestion.
- Webhooks.
- External-to-code linking.

Phase 10 — CLI/SDK

- CLI commands.
- Typed SDK.
- CI JSON output/exit codes.
- Example GitHub Action.

Phase 11 — Hardening

- Security audit.
- Load testing.
- Failure recovery.
- Backups.
- Observability.
- Rate limits/quotas.
- Large repository tests.
- Documentation.

Phase 12 — v1 release

- Release candidate.
- Demo repository.
- Screenshots/GIF/video.
- v1 documentation.
- GitHub release.
- Roadmap and issue templates.
- Community announcement.

28. Recommended Development Sequence

Foundation → Auth/Workspace → Repository → Git → AST → Symbols → Graph → Code DNA → Risk/Impact → Historical Evidence → AI/RAG → GitHub → CLI/SDK → Security → Observability → Documentation → Release.

Do not start with the chatbot. The graph and evidence engine are the product moat; the AI layer is the interface over that foundation.

29. Effort Estimate

For one experienced full-stack developer, a realistic first production-quality release is approximately **12–20 weeks** of consistent work. A polished multi-language ecosystem can take **6–12+ months**. These are planning estimates, not guarantees.

Workstream	Indicative effort
Foundation/auth/workspaces	1–2 weeks
Git/repository ingestion	2–3 weeks
AST/parser	2–4 weeks
Graph/impact	2–3 weeks
Code DNA/risk	1–2 weeks
AI/RAG	2–4 weeks
Frontend	2–4 weeks overlapping
GitHub/CLI/SDK	2–3 weeks overlapping
Testing/security/docs	2–4 weeks overlapping

30. Definition of Done — v1

- Fresh checkout starts using documented commands.
- Repository connects without manual DB work.
- Full and incremental analysis work.
- Failed jobs can retry/resume safely.
- TS/JS symbols and dependencies are queryable.
- Git history links to code entities with confidence.
- Architecture graph is interactive and filterable.
- Code DNA exposes history/metrics/risk/evidence.
- Impact analysis is deterministic.
- AI investigation provides citations/confidence and admits insufficient evidence.
- Private credentials are protected.
- RBAC and workspace isolation are tested.
- Operational failures are observable.
- CLI/API are documented.
- CI passes lint/type/test/build/security checks.
- License, README, CONTRIBUTING and SECURITY are complete.

31. Risks & Mitigations

Risk	Mitigation
Parser complexity	TS/JS first; adapter contract; fixtures; graceful degradation
History-to-symbol mapping imperfect	Confidence scores; line/file evidence; never claim certainty

Risk	Mitigation
AI hallucination	Evidence-first retrieval, citations, benchmark, unsupported-claim policy
Huge repositories	Incremental analysis, limits, ignore rules, bounded concurrency
Graph too large for UI	Aggregation, filters, depth limits, lazy expansion
Private source leakage	Local-first AI, secret isolation, explicit provider policy
Scope explosion	Strict v1 cutoff and staged language/integration roadmap

32. Post-v1 Roadmap

- Python/Java/Go/PHP/C#/Rust adapters.
- GitLab and Bitbucket.
- IDE extension.
- Runtime/observability evidence.
- Custom architecture rules.
- ADR generation.
- Cross-repository dependency intelligence.
- Repository comparison.
- Team knowledge-transfer reports.
- Historical change-risk prediction.
- Human-approved code-change proposals.
- Plugin ecosystem/marketplace.
- Optional hosted SaaS built from the same open-source core.

33. Final Product Vision

The long-term goal is a **historical intelligence layer for software**: code, structure, Git history, people, changes and engineering context become one evidence graph. Developers can interrogate the software system as an evolving artifact and understand both its present behavior and the history that produced it.

Final build checklist

1. Create the monorepo and infrastructure. **2.** Implement auth/workspaces. **3.** Build repository/Git ingestion. **4.** Build TS/JS AST and symbols. **5.** Build the graph. **6.** Add Code DNA and risk. **7.** Add deterministic impact analysis. **8.** Build historical evidence. **9.** Add Ollama/RAG investigation. **10.** Build the React explorer/graph. **11.** Add GitHub. **12.** Add CLI/SDK. **13.** Harden security/testing/operations. **14.** Document everything. **15.** Release v1 and grow through community contributions.

End of build-ready scope • Version 1.0